

The Scripting Boundary: Low-Level FFI Blueprints, QuickJS Memory Lifecycles, and Deterministic Intent Payloads for WebAssembly GSG Engines

1. Architectural Mandate and The FFI Boundary

The engineering of a browser-native, macroscopic Grand Strategy Game (GSG) demands an architecture that simultaneously satisfies the rigid constraints of a compiled WebAssembly (WASM) linear memory space and the dynamic flexibility required by a macroscopic modding API. The integration of a native C++/Rust Entity-Component-System (ECS) with an embedded QuickJS interpreter and an Off-Main-Thread (OMT) Svelte 5 user interface introduces profound friction points at the scripting boundaries.

The primary architectural challenge lies in managing the Foreign Function Interface (FFI). Crossing the boundary between the WASM linear memory heap and the QuickJS runtime heap—or communicating across the thread boundary from the Svelte 5 UI worker to the WASM worker—incurs significant performance penalties if not strictly orchestrated. In a deterministic lockstep simulation executing thousands of entities per tick, traditional paradigms of JSON serialization, closure passing, and dynamic type resolution are computationally unviable.

This technical blueprint explicitly delineates the data structures, memory lifecycles, and FFI binding signatures required to bridge these domains. The analysis focuses strictly on three intersection points: the translation of JavaScript mathematical modifier logic into native C++ Directed Acyclic Graph (DAG) opcodes, the precise reference-counting memory lifecycle of deferred QuickJS AI callbacks within the C++ temporal scheduler, and the bitwise-packed binary serialization format of the Svelte 5 Intent Queue over a SharedArrayBuffer.

2. QuickJS to C++ DAG Integration: The FFI Payload & Lifecycle

To support a deeply emergent modding ecosystem, modders must be able to define custom mathematical modifiers using JavaScript or TypeScript. These modifiers (e.g., "National Tax +5% if At War") attach to specific entities within the ECS and feed into the global math aggregation pipeline. The core engineering problem is passing the evaluation logic of these modifiers across the QuickJS-to-C++ FFI boundary without passing executable JavaScript closures. Executing a QuickJS closure for every active modifier during the C++ read-lock phase would trigger

thousands of boundary crossings per tick, instantly stalling the fixed temporal accumulator.

2.1 The Postfix Opcode Serialization Strategy

To prevent FFI bottlenecking, the architecture implements a strict boundary separation between *definition* and *evaluation*. The QuickJS environment is utilized solely to parse and define the modifier blueprint during game initialization or mod loading. The actual mathematical evaluation is executed natively within the C++ Directed Acyclic Graph (DAG).³

When a modder writes a modifier expression, the JavaScript API compiles this mathematical string into a Reverse Polish Notation (RPN) array of integer opcodes and operands. This technique, heavily utilized in embedded expression evaluators, flattens a complex Abstract Syntax Tree (AST) into a linear array of instructions that a lightweight C++ stack machine can evaluate natively.⁵

For example, the expression $(\text{BaseTax} * 1.05) + \text{FlatBonus}$ is serialized within QuickJS into a typed array of 32-bit integers: ``.

This highly compact binary array is passed across the FFI boundary exactly once. The C++ engine ingests this array and constructs a native DAG node containing the opcode sequence. Consequently, during the ECS read-lock phase, the C++ engine traverses the DAG and evaluates the opcodes natively utilizing its fixed-point integer mathematics, entirely bypassing the QuickJS interpreter.

2.2 Memory Layout and Cache-Line Alignment of the DAG Node

The native C++ struct representing the DAG node must be meticulously aligned to ensure cache-line density within the WASM linear memory.⁸ The struct encapsulates the node's topological identifier, its dirty-flag state, the cached deterministic value, and a pointer to the ingested opcode array.

C++

```
#include <cstdint>
#include <new>
```

```
#pragma pack(push, 4) // Force 4-byte alignment for WASM linear memory optimization
struct DAGNodePayload {
    uint32_t node_id;    // Topologically sorted index within the global DAG
    uint32_t flags;      // Bitfield: Bit 0 = IsDirty, Bit 1 = IsActive
    int32_t cached_val;  // The deterministic fixed-point evaluation result
    uint32_t opcode_length; // Length of the instruction array
    int32_t* opcode_stream; // Pointer to the dynamically allocated RPN instructions
}
```

```
};  
#pragma pack(pop)
```

The use of `#pragma pack(push, 4)` is a critical directive when compiling to WebAssembly. While WebAssembly intrinsically handles 32-bit and 64-bit values¹⁰, enforcing strict 4-byte alignment on aggregate members prevents the LLVM compiler (via Emscripten) from inserting arbitrary padding bytes, thereby reducing the overall memory footprint of the DAG array and ensuring that sequential reads during the DAG traversal remain perfectly localized within CPU L1/L2 caches.⁸

2.3 The Thin Accessor Paradigm and FFI Binding Signatures

While the evaluation is native, the modder's JavaScript code still requires a reference to the modifier to toggle its state or query its value asynchronously. To facilitate this, the QuickJS runtime exposes a "Thin Accessor"—a lightweight JavaScript object that acts as a proxy to the native C++ `DAGNodePayload`.

This proxy object is established by registering a custom class ID within the QuickJS runtime using `JS_NewClassID`.¹²

C++

```
static JSClassID js_modifier_class_id;  
  
void InitializeModifierClass(JSRuntime* rt) {  
    JS_NewClassID(rt, &js_modifier_class_id); // Allocates a globally unique Class ID  
  
    JSClassDef def = {};  
    def.class_name = "ModifierDAGNode";  
    def.finalizer = js_modifier_finalizer; // Crucial for memory lifecycle  
  
    JS_NewClass(rt, js_modifier_class_id, &def);  
}
```

When the QuickJS environment calls the native function to construct a new modifier, the C++ FFI binding performs the memory allocation, ingests the opcode array, and securely binds the native C++ pointer to the QuickJS object using the `JS_SetOpaque` function.¹²

C++

```

static JSValue js_create_modifier_node(JSContext *ctx, JSValueConst this_val, int argc, JSValueConst
*argv) {
    if (argc < 2) return JS_ThrowTypeError(ctx, "Requires string ID and opcode Int32Array.");

    // 1. Extract the Int32Array buffer from QuickJS
    size_t byte_length;
    uint8_t* js_buffer = JS_GetArrayBuffer(ctx, &byte_length, argv);
    uint32_t opcode_count = byte_length / sizeof(int32_t);

    // 2. Allocate native C++ DAG node aligned to 16-byte boundaries for SIMD safety
    DAGNodePayload* native_node = new (std::align_val_t(16)) DAGNodePayload();
    native_node->flags = 0x01; // Initialize as Dirty
    native_node->opcode_length = opcode_count;

    // 3. Perform a zero-copy ingestion or deep copy of the opcode stream
    native_node->opcode_stream = new int32_t[opcode_count];
    std::memcpy(native_node->opcode_stream, js_buffer, byte_length);

    // 4. Create the Thin Accessor JS object bound to the custom ClassID
    JSValue obj = JS_NewObjectClass(ctx, js_modifier_class_id);
    if (JS_IsException(obj)) {
        delete native_node->opcode_stream;
        delete native_node;
        return obj;
    }

    // 5. Bind the native C++ pointer to the QuickJS object's opaque data field [12, 14]
    JS_SetOpaque(obj, native_node);

    return obj;
}

```

The resulting JSValue returned to the JavaScript environment is devoid of complex closures or heavy structural properties. It is merely a shell containing an opaque pointer. Whenever the JavaScript script attempts to read the modifier's value, it does so through a property getter defined by JS_CGETSET_DEF¹⁵, which retrieves the opaque pointer via JS_GetOpaque and returns the native cached_val.

2.4 Dirty-Flag Invalidation Routing Across the Boundary

The C++ DAG utilizes a lazy-evaluation "dirty-flag" caching mechanism. If a base entity stat changes, the DAG propagates a dirty flag downwards to all dependent modifier nodes. A

critical synchronization risk occurs if the QuickJS script attempts to read a modifier value exactly when it is flagged as dirty but before the C++ engine has reached it in the sequential DAG evaluation pass.

To ensure deterministic integrity, the QuickJS Thin Accessor getter must actively participate in the dirty-flag resolution. The getter must inspect the native C++ flags before returning the value.¹²

C++

```
static JSValue js_modifier_get_value(JSContext *ctx, JSValueConst this_val) {
    // Retrieve the native pointer bound to the JS object [12, 14]
    DAGNodePayload* native_node = static_cast<DAGNodePayload*>(JS_GetOpaque(this_val,
js_modifier_class_id));
    if (!native_node) return JS_ThrowInternalError(ctx, "Null DAG Node Reference");

    // Check Bit 0 for Dirty State
    if (native_node->flags & 0x01) {
        // Trigger localized synchronous evaluation of the RPN opcode array
        DAGSystem::ForceEvaluateNode(native_node);
        // ForceEvaluateNode will clear the dirty flag upon completion
    }

    // Return the deterministic fixed-point integer back to JS space
    return JS_NewInt32(ctx, native_node->cached_val);
}
```

By routing the read operation directly into the C++ dirty-flag evaluation logic, the system guarantees that the QuickJS environment never reads stale data, and it does so without maintaining duplicate cache states across the memory boundary.

2.5 Memory Lifecycle, Finalizers, and Tombstone Registries

The memory lifecycle mapping between a garbage-collected QuickJS environment and a manually managed C++ engine is inherently volatile. QuickJS utilizes a reference counting system combined with a mark-and-sweep cycle collector to manage heap allocations.¹⁶ Discrepancies between the QuickJS reference count and the C++ allocation state will inevitably lead to memory leaks or catastrophic use-after-free WASM traps.¹⁶

Scenario A: JavaScript discards the modifier. If the modder's script nullifies all references to the modifier object, the QuickJS reference count reaches zero.¹⁶ However, the native

DAGNodePayload remains allocated in the C++ heap. To resolve this memory leak, the architecture leverages the JSClassFinalizer defined during class registration.¹²

C++

```
static void js_modifier_finalizer(JSRuntime *rt, JSValue val) {
    DAGNodePayload* native_node = static_cast<DAGNodePayload*>(JS_GetOpaque(val,
js_modifier_class_id));
    if (native_node) {
        // Warning: It is invalid to execute JS code within a finalizer [12, 13, 14]
        // Warning: Do not delete immediately if the DAG is currently iterating (Concurrency hazard)

        // Push the node_id to a thread-safe deferred deletion queue
        DAGSystem::RequestDeferredDeletion(native_node->node_id);
    }
}
```

The finalizer acts as the bridge that notifies C++ of the JavaScript object's demise. The C++ engine, at the end of the lockstep frame (during the write-lock phase), drains the deferred deletion queue, safely detaches the node from the DAG topology, and executes `delete native_node->opcode_stream` followed by `delete native_node`.

Scenario B: C++ destroys the entity holding the modifier. Conversely, if an entity is destroyed in the simulation, the C++ engine must deallocate the DAGNodePayload. However, a QuickJS script may still hold a variable reference to the Thin Accessor object. If the JS script attempts to call `modifier.value`, `JS_GetOpaque` will return a dangling pointer, leading to a segmentation fault within the WASM sandbox.¹⁴

To mitigate this, the architecture implements a C++ Tombstone Registry. Instead of raw memory deallocation, `delete native_node` is replaced by a memory pooling system. When an entity dies, the C++ engine overwrites the DAGNodePayload memory block, setting a specific "Tombstone" signature in the flags field (e.g., `0xFFFFFFFF`).

Every FFI getter/setter must perform a tombstone check immediately after `JS_GetOpaque`:

C++

```
DAGNodePayload* native_node = static_cast<DAGNodePayload*>(JS_GetOpaque(this_val,
```

```
js_modifier_class_id));  
    if (native_node && native_node->flags == 0xFFFFFFFF) {  
        return JS_ThrowInternalError(ctx, "Attempted to access destroyed DAG Node (Tombstone).");  
    }
```

This bidirectional memory handshake ensures absolute cohesion between the QuickJS reference counter and the C++ DAG topology, precluding memory leaks and ensuring stable lockstep determinism.

3. Managing QuickJS AI Callbacks in the C++ Modulo Scheduler

In the macroscopic GSG, AI factions require the ability to evaluate complex strategic logic. To prevent CPU frame stalling, this logic cannot execute uniformly across all entities simultaneously. The C++ engine utilizes a modulo-based temporal bucket scheduler, which distributes AI logic execution sequentially across future lockstep ticks. Modders script these AI evaluation functions in JavaScript, introducing the complex requirement of passing a QuickJS callback across the FFI boundary, storing it in a C++ task queue, and executing it deterministically at a deferred temporal moment.

3.1 Deferred Execution Constraints in a Deterministic Context

Standard JavaScript execution relies on the browser's native Event Loop and micro-task queues to handle asynchronous callbacks (e.g., `setTimeout`, `Promises`).¹⁸ In a deterministic lockstep environment, the native browser Event Loop is fundamentally incompatible. Asynchronous execution must be strictly governed by the internal temporal accumulator of the C++ engine.

When a modder registers a deferred AI function using `AIEngine.deferEvaluation(entityId, tickDelay, function() {...})`, the QuickJS environment executes the registration synchronously. If the C++ FFI layer simply stores a raw pointer to this `JSValue` function, the QuickJS garbage collector will immediately destroy the function closure as soon as the registration scope ends, because there are no active JavaScript variables holding a reference to it. When the C++ modulo scheduler reaches the target tick and attempts to execute the stored pointer, it will attempt to evaluate freed memory, triggering an unrecoverable WASM exception.¹⁶

3.2 The C++ Task Queue Memory Blueprint

To safely bridge the temporal gap, the C++ engine implements a specific structural blueprint for the deferred task payload. This struct encapsulates the execution context, the target entity, and the actual QuickJS function handles.

C++

```
struct JS_AI_DeferredTask {  
    JSContext* ctx;      // The specific QuickJS realm context  
    JSValue  callback;   // The JS function to execute (The AI logic)  
    JSValue  this_bind;  // The 'this' execution context (Entity Handle)  
    uint32_t entity_id;  // ECS entity target for validation  
    uint32_t target_tick; // The precise lockstep tick for scheduled execution  
};
```

The C++ modulo scheduler organizes these tasks using a statically sized array of vectors representing the temporal buckets:

```
std::vector<JS_AI_DeferredTask> temporal_buckets;
```

3.3 Reference Pinning Mechanics via JS_DupValue

To guarantee the survival of the JavaScript function closure until the C++ modulo scheduler determines it is time to execute, the C++ engine must explicitly seize memory ownership of the JSValue.

QuickJS manages heap memory via a strict reference counting algorithm.¹⁶ A JSValue is only freed when its internal ref_count integer reaches zero. The QuickJS C API exposes the JS_DupValue function specifically to allow native code to increment this reference count.¹⁴

When the FFI registration endpoint receives the callback arguments, it must execute a manual "pinning" strategy.

C++

```
static JSValue js_register_ai_task(JSContext *ctx, JSValueConst this_val, int argc, JSValueConst *argv) {  
    if (argc < 3) return JS_ThrowTypeError(ctx, "Invalid arguments for AI deferral.");  
  
    uint32_t entity_id, target_tick;  
    JS_ToUint32(ctx, &entity_id, argv);  
    JS_ToUint32(ctx, &target_tick, argv);  
    JSValueConst callback = argv;  
  
    if (!JS_IsFunction(ctx, callback)) return JS_ThrowTypeError(ctx, "Argument 3 must be a
```

```
function.");

    JSAlDeferredTask task;
    task.ctx = ctx;
    task.entity_id = entity_id;
    task.target_tick = target_tick;

    // CRITICAL PINNING PHASE: Increment the reference count of the JS values
    task.callback = JS_DupValue(ctx, callback);
    task.this_bind = JS_DupValue(ctx, this_val);

    // Insert the pinned task into the native C++ modulo scheduler
    uint32_t bucket_index = target_tick % BUCKET_COUNT;
    temporal_buckets[bucket_index].push_back(task);

    return JS_UNDEFINED;
}
```

By executing `JS_DupValue`, the C++ `std::vector` becomes an explicit root owner of the function. The QuickJS internal `ref_count` is incremented, and the garbage collector will categorically ignore the object, rendering it immune to collection regardless of whether JavaScript drops all references to it.¹⁶

3.4 Execution, Exception Handling, and the Unpinning Lifecycle

At the exact onset of the targeted lockstep tick, the C++ engine must drain the corresponding temporal bucket. The lifecycle of the execution phase requires meticulous handling of the invocation, exception containment, and the mandatory memory cleanup to prevent persistent leaks.

The invocation of the pinned callback is performed using `JS_Call`.¹⁸ Because the execution is occurring during the highly parallelized C++ read-lock phase, the engine must gracefully trap any JavaScript exceptions. If a modder's script throws an error, an unhandled exception propagating back to C++ would immediately halt the deterministic simulation.

C++

```
void DrainTemporalBucket(uint32_t current_tick) {
    uint32_t bucket_index = current_tick % BUCKET_COUNT;
    auto& bucket = temporal_buckets[bucket_index];
```

```

for (auto& task : bucket) {
    if (task.target_tick!= current_tick) continue; // Lockstep safety assertion

    // Construct an argument array to pass context (e.g., current tick) into JS
    JSValue arg = JS_NewInt32(task.ctx, current_tick);
    JSValueConst args = { arg };

    // Execute the pinned callback within the QuickJS VM
    JSValue result = JS_Call(task.ctx, task.callback, task.this_bind, 1, args);

    // Exception Containment Protocol [20, 21]
    if (JS_IsException(result)) {
        JSValue exception_val = JS_GetException(task.ctx);
        // Engine logs the exception internally to prevent crash
        LogWASMException(task.ctx, exception_val);
        JS_FreeValue(task.ctx, exception_val); // Must free the exception object
    }

    // CRITICAL UNPINNING PHASE: Releasing the explicitly held memory
    JS_FreeValue(task.ctx, result); // Free the return value
    JS_FreeValue(task.ctx, arg); // Free the constructed argument
    JS_FreeValue(task.ctx, task.callback); // Decrement reference count of the callback
    JS_FreeValue(task.ctx, task.this_bind); // Decrement reference count of the context
}

// Clear the executed bucket to finalize the lifecycle
bucket.clear();
}

```

The invocation of `JS_FreeValue` on `task.callback` is the crucial unpinning step. This explicitly decrements the reference count. Assuming the JavaScript environment no longer holds a reference to the anonymous callback function, the reference count falls to zero, and the QuickJS engine immediately deallocates the memory.¹⁶ Failure to call `JS_FreeValue` on every pinned object in the queue creates a cumulative memory leak that will eventually trigger a WASM Out-Of-Memory (OOM) trap.

3.5 Cyclic Reference Mitigation and Tombstone Purging

While reference counting is fast and deterministic, it is inherently vulnerable to cyclic references. QuickJS employs a Bacon-based cyclic collection algorithm to resolve these cycles.¹⁷ The Bacon cycle collector identifies subgraphs of the QuickJS heap where reference counts are entirely accounted for by internal linkages, proving the subgraph is an isolated cycle, and thus safe to collect.²²

However, the Bacon algorithm is fundamentally blind to C++ memory structures. It cannot traverse the C++ `std::vector` inside the modulo scheduler. If an AI callback closure captures a reference to the `this_bind` object (the Entity Accessor), and that Entity Accessor is pinned in the C++ queue, an unbreakable cross-boundary cycle is formed.

A severe memory failure scenario arises if the simulated entity is destroyed *before* its targeted temporal tick arrives. The ECS will delete the entity, but the `JSAIDeferredTask` remains pinned inside the temporal bucket, holding an active reference count to the QuickJS objects via `JS_DupValue`. The QuickJS cycle collector cannot see that the C++ entity is dead, and the C++ engine will wait until the target tick to free the JS memory. If thousands of entities die in a massive battle, the delayed unpinning creates a catastrophic memory bloat spike.

To mitigate this, the architecture mandates an explicit Tombstone Purging pass. When the ECS destroys an entity, it broadcasts a termination signal to the AI scheduler. The scheduler must proactively traverse the temporal buckets, locate any deferred tasks belonging to the destroyed entity_id, explicitly unpin the memory, and erase the struct.

C++

```
void PurgeDeadEntityTasks(uint32_t dead_entity_id) {
    for (uint32_t i = 0; i < BUCKET_COUNT; ++i) {
        auto& bucket = temporal_buckets[i];

        // Utilize the erase-remove idiom for safe vector mutation
        bucket.erase(std::remove_if(bucket.begin(), bucket.end(),
            [dead_entity_id](JSAIDeferredTask& task) {
                if (task.entity_id == dead_entity_id) {
                    // Manually unpin the JS resources immediately
                    JS_FreeValue(task.ctx, task.callback);
                    JS_FreeValue(task.ctx, task.this_bind);
                    return true; // Mark for erasure
                }
            }), bucket.end());
    }
}
```

This manual purging mechanism guarantees that the C++ task queue actively manages the cyclic lifecycle, preventing cross-boundary deadlocks and ensuring the WASM container maintains a stable memory footprint over long simulation durations.

4. Svelte 5 Intent Queue: The Binary Payload Structure

The separation of the Svelte 5 User Interface into an Off-Main-Thread (OMT) Web Worker architecture isolates UI rendering latency from the deterministic lockstep simulation. However, the UI cannot execute direct mutations on the game state. It must serialize human interactions into "Intent Payloads" that are queued and subsequently ingested by the WASM engine at the exact commencement of the next fixed tick.

Traditional approaches to Web Worker messaging rely on `postMessage` with structured cloning, which serializes data into complex JSON-like objects.²³ In a highly dense GSG, passing object graphs across the thread boundary introduces unacceptable allocation overhead, non-deterministic string parsing latency, and triggers massive V8 Garbage Collection pauses that stall UI rendering.²³

To achieve absolute zero-copy memory bridging, the architecture implements a tightly packed, lockstep-safe ring buffer operating directly over a `SharedArrayBuffer`.² This design dictates a highly optimized bitwise binary payload structure that the C++ engine can parse in a single linear sweep without ever instantiating strings or dynamically typed objects.²⁶

4.1 The Wait-Free Ring Buffer Blueprint

The communication bridge is a fixed-size contiguous block of memory allocated as a `SharedArrayBuffer`. Both the JavaScript Svelte Worker and the WASM C++ environment construct a `TypedArray` view—specifically an `Int32Array`—over this shared memory block.²⁴

To facilitate safe, lock-free concurrency between the UI thread (producer) and the WASM thread (consumer), the architecture relies on a Wait-Free Ring Buffer pattern utilizing atomic operations. The first segment of the buffer is strictly reserved for synchronization headers.

Memory Offset (Bytes)	View (Int32Array)	Structural Purpose	Atomic Access Rule
0x00 - 0x03	Index	Head Index: Tracks ingestion progress.	Read by JS, Written by C++ via <code>__atomic_store</code>
0x04 - 0x07	Index	Tail Index: Tracks payload insertion.	Read by C++, Written by JS via <code>Atomics.store</code>

0x08 - 0x0B	Index	Overflow Flag: Diagnostic state indicator.	Written by JS on buffer exhaustion
0x0C - 0x0F	Index	Padding: Enforces 16-byte alignment.	Reserved
0x10 - Max	Index [4...N]	Payload Ring: Contiguous data structs.	Sequentially written by JS, Read by C++

The Svelte UI utilizes the Atomics API to safely mutate the Tail Index after writing an intent. At the start of the lockstep tick, the C++ engine reads the Tail Index atomically and processes all commands sequentially until the Head Index matches the Tail Index, finalizing the ingestion by updating the Head.

4.2 Bitwise Packing Strategy for Complex Actions

To eliminate the need for parsing string-based opcodes, every conceivable human interaction is mapped to a standardized, integer-based binary format. While simple commands can fit into a single 32-bit integer, complex macroscopic commands (e.g., declaring war with specific casus belli parameters, or queuing construction arrays) require a multi-byte structured format.

To maintain Int32Array synchronization, all commands are standardized into fixed-width 8-byte (64-bit) blocks. This allows the UI to pass vast arrays of data using simple bitwise shift operations, entirely avoiding object allocation.²⁶

Consider the serialization of a complex building order: *"Build a Level 3 Farm at Province ID 4502 using National Focus Tree Buff 12"*.

Instead of { action: "build", type: 4, level: 3, target: 4502, focus: 12 }, the Svelte application executes a highly efficient bitwise packing algorithm to construct two 32-bit integers representing the 8-byte payload.

The 8-Byte Command Structure:

- **Word 0 (First 32 Bits): The Action Header**
 - Bits 0-15: Target ID (e.g., Province ID 4502)
 - Bits 16-23: Entity Type ID (e.g., Farm = 4)
 - Bits 24-31: Opcode (e.g., ACTION_BUILD = 0x02)

- **Word 1 (Second 32 Bits): The Action Parameters**

- Bits 0-7: Level/Tier (e.g., Level 3)
- Bits 8-15: National Focus ID (e.g., Focus 12)
- Bits 16-31: Reserved for future padding

The JavaScript serialization logic utilizes bitwise unsigned right shifts ($\gg$) and bitwise OR ($|$) to pack the payload securely into the array view ²⁶:

JavaScript

```
// Svelte 5 OMT Worker: Intent Dispatcher
function dispatchBuildIntent(targetId, typeId, level, focusId) {
  const OPCODE_BUILD = 0x02;

  // Word 0 Construction
  // Opcode shifted 24 bits, Type shifted 16 bits, Target occupies lowest 16
  const word0 = (OPCODE_BUILD << 24) | (typeId << 16) | targetId;

  // Word 1 Construction
  // Focus shifted 8 bits, Level occupies lowest 8
  const word1 = (focusId << 8) | level;

  // Atomic Ring Buffer Insertion
  const tail = Atomics.load(sharedBufferView, 1);
  const nextTail = (tail + 2) % MAX_QUEUE_SIZE;

  // Ensure we don't overrun the C++ Head pointer
  const head = Atomics.load(sharedBufferView, 0);
  if (nextTail === head) {
    Atomics.store(sharedBufferView, 2, 1); // Set Overflow Flag
    return;
  }

  // Write the 8-byte packed block starting at index 4 (offsetting headers)
  const payloadStart = 4 + tail;
  sharedBufferView = word0;
  sharedBufferView = word1;

  // Publish the atomic update to notify C++
  Atomics.store(sharedBufferView, 1, nextTail);
}
```

```
}
```

This procedure generates zero garbage collection overhead. The entire complex action is serialized via immediate CPU registers and written directly into shared linear memory in sub-microsecond latency.

4.3 WASM Memory Alignment and Structural Pragma Directives

On the native side, the C++ engine must ingest this continuous stream of 32-bit integers. Reading raw bitmasks via repeated shift operators in C++ is tedious and error-prone. Instead, the architecture maps an exact C++ structure directly over the memory pointer provided by the `SharedArrayBuffer`.⁸

However, mapping C++ structs to raw byte arrays in WebAssembly introduces severe alignment skew risks. The LLVM compiler natively applies data structure alignment (padding) to optimize CPU fetch cycles. If a struct contains a `uint8_t` followed by a `uint16_t`, the compiler implicitly inserts a padding byte, causing the C++ struct to map incorrectly to the tightly packed JavaScript `Int32Array`.⁸

To achieve guaranteed zero-copy ingestion, the structural blueprint employs `#pragma pack` directives to forcibly strip all compiler padding, aligning the bitfields identically to the Svelte bitwise offsets.⁸

```
C++
```

```
#include <stdint>
```

```
#pragma pack(push, 1) // Enforce absolute 1-byte packing boundaries
```

```
union IntentPayload {
```

```
    // Allows raw traversal in 32-bit blocks
```

```
    int32_t raw_words;
```

```
    // Explicit mapping of Word 0 and Word 1 bitfields
```

```
    struct {
```

```
        // Word 0 (Mapped from right to left based on Little Endian architecture)
```

```
        uint32_t target_id : 16;
```

```
        uint32_t entity_type : 8;
```

```
        uint32_t opcode : 8;
```

```
        // Word 1
```

```
        uint32_t level : 8;
```

```
        uint32_t focus_id : 8;
```

```

    uint32_t reserved : 16;
} build_cmd;

struct {
    uint32_t source_id : 16;
    uint32_t target_id : 16;

    uint32_t stance : 8;
    uint32_t modifiers : 24;
} combat_cmd;
};
#pragma pack(pop)

```

The union guarantees that the IntentPayload takes exactly 8 bytes (64 bits) in linear memory. This ensures a flawless 1:1 structural overlap between the JavaScript memory array and the native C++ parser logic.²⁸

4.4 Sequential Lockstep Ingestion and Atomic Synchronization

At the inception of the fixed C++ lockstep tick, prior to the read-lock phase, the engine must atomically drain the Intent Queue. The wait-free ingestion loop utilizes GNU atomic intrinsic functions (`__atomic_load_n` and `__atomic_store_n`) to ensure memory barrier correctness without utilizing heavy OS-level mutexes, which are highly inefficient in a single-threaded WASM context.

C++

```

void ProcessIntentQueue(int32_t* shared_buffer) {
    // 1. Atomically read the synchronization headers
    uint32_t head = __atomic_load_n(&shared_buffer, __ATOMIC_SEQ_CST);
    uint32_t tail = __atomic_load_n(&shared_buffer, __ATOMIC_SEQ_CST);

    // 2. Cast the payload section (skipping the 4 integer headers) to our packed union
    IntentPayload* queue_data = reinterpret_cast<IntentPayload*>(&shared_buffer);

    // 3. Linear wait-free sequential drain
    while (head != tail) {
        // Since payload size is 8-bytes, we advance head by 2 integers per iteration
        uint32_t payload_index = head / 2;
        IntentPayload payload = queue_data[payload_index];
    }
}

```

```

// The Opcode is automatically extracted by the bitfield struct mapping
switch (payload.build_cmd.opcode) {
    case 0x02: // ACTION_BUILD
        GlobalState::ExecuteBuild(
            payload.build_cmd.target_id,
            payload.build_cmd.entity_type,
            payload.build_cmd.level,
            payload.build_cmd.focus_id
        );
        break;
    case 0x03: // ACTION_COMBAT
        GlobalState::ExecuteCombat(
            payload.combat_cmd.source_id,
            payload.combat_cmd.target_id,
            payload.combat_cmd.stance
        );
        break;
    // Additional Opcode routes...
}

// Advance the head pointer by 2 (the length of the 8-byte payload block)
head = (head + 2) % MAX_QUEUE_SIZE;
}

// 4. Atomically publish the updated Head, finalizing the consumption [24]
__atomic_store_n(&shared_buffer, head, __ATOMIC_SEQ_CST);
}

```

By designing the intent payload as a strictly aligned C++ struct overlaid on a wait-free SharedArrayBuffer ring, the architecture achieves the pinnacle of cross-boundary performance. The system translates tens of thousands of complex UI interactions directly into native CPU instructions with absolutely zero memory allocation, zero string parsing, and perfect determinism suitable for lockstep network transmission.

5. Architectural Synthesis

The engineering of the scripting boundary in a compiled WASM ecosystem demands an unyielding adherence to memory management fundamentals. By shifting mathematical definitions into compiled RPN opcode arrays, the QuickJS runtime is relieved of computational overhead, operating purely as a Thin Accessor proxy. By actively manipulating the QuickJS reference counters via JS_DupValue and JS_FreeValue¹⁴, the C++ temporal scheduler achieves

flawless deferred execution without violating the strict rules of the Bacon cyclic collector.²² Finally, by enforcing absolute bitwise synchronization between Svelte Int32Array construction and C++ `#pragma pack` directives⁸, the UI maintains a zero-allocation, wait-free dominance over the FFI boundary. This triad of low-level FFI blueprints establishes the resilient infrastructural bedrock necessary for a deterministic, browser-native macroscopic simulation.

Works cited

1. SharedArrayBuffer: The Hidden Super-Primitive That's Reshaping the Future of WebAssembly, .NET & Parallel Runtime Architecture | by Jacob Mellor | Medium, accessed April 19, 2026, <https://medium.com/@jacobscottmellor/sharedarraybuffer-the-hidden-super-primitive-thats-reshaping-the-future-of-webassembly-net-e369e667f6e9>
2. Fluxins is a lightweight expression parser and evaluator written in modern C++. - GitHub, accessed April 19, 2026, <https://github.com/anstropleuton/fluxins>
3. C++ Mathematical Expression Library (ExprTk) - By Arash Partow, accessed April 19, 2026, <https://www.partow.net/programming/exprtk/>
4. How do I optimize this postfix expression tree for speed? - Stack Overflow, accessed April 19, 2026, <https://stackoverflow.com/questions/2772817/how-do-i-optimize-this-postfix-expression-tree-for-speed>
5. Why do compilers typically convert code into abstract syntax/parse trees before the final product?, accessed April 19, 2026, <https://langdev.stackexchange.com/questions/3662/why-do-compilers-typically-convert-code-into-abstract-syntax-parse-trees-before>
6. christian-vigh/eval: An infix expression evaluator easily embeddable in C/C++ programs - GitHub, accessed April 19, 2026, <https://github.com/christian-vigh/eval>
7. #pragma pack - IBM, accessed April 19, 2026, <https://www.ibm.com/docs/en/zos/2.5.0?topic=descriptions-pragma-pack>
8. pack pragma | Microsoft Learn, accessed April 19, 2026, <https://learn.microsoft.com/en-us/cpp/preprocessor/pack?view=msvc-170>
9. Types — WebAssembly 3.0 (2026-04-07), accessed April 19, 2026, <https://webassembly.github.io/spec/core/syntax/types.html>
10. Web Evolution and WebAssembly, accessed April 19, 2026, <https://www.cs.mcgill.ca/~cs520/2018/slides/17-web-assembly.pdf>
11. Custom Classes - QuickJS-ng - Mintlify, accessed April 19, 2026, <https://www.mintlify.com/quickjs-ng/quickjs/api/objects/classes>
12. quickjs/doc/quickjs.texi at master · bellard/quickjs - GitHub, accessed April 19, 2026, <https://github.com/bellard/quickjs/blob/master/doc/quickjs.texi>
13. The QuickJS C API - GitHub Pages, accessed April 19, 2026, <https://quickjs-ng.github.io/quickjs/developer-guide/intro>
14. Use QuickJS to modify preferences and global variables - Stack Overflow, accessed April 19, 2026, <https://stackoverflow.com/questions/70123636/use-quickjs-to-modify-preferences-and-global-variables>

15. Reference Counting - QuickJS-ng - Mintlify, accessed April 19, 2026,
<https://www.mintlify.com/quickjs-ng/quickjs/api/values/reference-counting>
16. Anatomy of QuickJS garbage collection algorithm | by Landerl Young - Medium, accessed April 19, 2026,
<https://medium.com/@landerlyoung/anatomy-of-quickjs-garbage-collection-algorithm-fc02f6813ba1>
17. Building a Runtime with QuickJS - Andrew Healey, accessed April 19, 2026,
<https://healeycodes.com/building-a-runtime-with-quickjs>
18. Understanding JavaScript's Task(Callback) and Micro-task Queues | by Sibin George, accessed April 19, 2026,
<https://medium.com/@sibingeorge2698/understanding-javascripts-task-callback-and-micro-task-queues-47e9fb00a7a4>
19. Apparently the garbage collection algorithm is reference counting with cycle rem... | Hacker News, accessed April 19, 2026,
<https://news.ycombinator.com/item?id=20413561>
20. Passing structured data between C++ and JavaScript in Web Assembly - DEV Community, accessed April 19, 2026,
<https://dev.to/azure/passing-structured-data-from-c-to-javascript-in-web-assembly-1iOp>
21. ArrayBuffer in JavaScript: A Deep Dive into Binary Data Handling (Part 2) | by Dipta Barua, accessed April 19, 2026,
<https://medium.com/@diptab11/arraybuffer-in-javascript-a-deep-dive-into-binary-data-handling-part-2-e97862df799a>
22. Transferring array of C structs to main JS thread with zero-copy via SharedArrayBuffer #22007 - GitHub, accessed April 19, 2026,
<https://github.com/emscripten-core/emscripten/discussions/22007>
23. Passing arrays between wasm and JavaScript - rob-blackbourn.github.io, accessed April 19, 2026,
<https://rob-blackbourn.github.io/blog/2020/06/07/wasm-arrays/>
24. Zero-copy pass ArrayBuffer from JS-land to WebAssembly-land · Issue #1162 - GitHub, accessed April 19, 2026,
<https://github.com/WebAssembly/design/issues/1162>
25. How to deal with alignment skew when storing a structure in a byte buffer? - Stack Overflow, accessed April 19, 2026,
<https://stackoverflow.com/questions/71838002/how-to-deal-with-alignment-skew-when-storing-a-structure-in-a-byte-buffer>